

FINDS: An N-Body Simulator for Computing the Evolution of Large Schools

Mufaro Machaya¹, Mohamed Niged Mabrouk², and Daniel Floryan² *

August 2026

¹University of British Columbia

²University of Houston

Abstract

Animals travelling in cohesive groups yields many collective and individual benefits, such as security against predators and energy efficiency. The replication of such benefits for analogous, biology-inspired technologies like autonomous drone systems requires keen understanding of the cohesion and evolution of systems of these groups, but these investigations are fundamentally limited to small sizes due to the high computational complexity in computing pairwise dynamics at order N^2 . As real schools of fish have been recorded to contain millions of members, FINDS, or Fish INteraction and Dynamics Simulator, was created to efficiently simulate the dynamics of very large schools of fish using common computational techniques employed in N-body simulation. FINDS is implemented as a containerized, modular software suite written predominantly in C (with Python for post-processing) that handles three core simulation processes in order. First, FINDS contains routines for computing the time-derivative of a school, which is a function of the sum of the pairwise interactions for each member of the school. As such, FINDS contains implementations of Brute Force computation, the Barnes-Hut algorithm, and the Fast Multipole Method. Second, FINDS contains custom routines for numerically integrating these systems using adaptive and non-adaptive Runge-Kutta schemes for adaptive time-stepping. Third, FINDS contains a small library of post-processing modules written in Python for stream-lining the analysis of generated data.

Introduction

Synchronous mass movement is a common pattern throughout nature, observed most commonly in the movement of smaller animals or prey animals such as birds, fish, bugs, oxen, deer, etc. due to the multitude of benefits that members of such groups gain, such as increased protection from predators, increased energy efficiency due to the motion of others (such as vortical wake), etc. *finish later!!!!!!*

Literature Review and Similar Work

Mathematical Model of Swimmers

FINDS is developed around the far-field source/sink dipole model designed by Mabrouk and Floryan for modelling how passive hydrodynamic interactions at far distances effects the evolution of a greater school [1, 2]. Under this model, each fish i is defined as a dipole consisting of a fluid source at the head of the fish $\mathbf{x}_{f,i}$, and a fluid sink at the tail $\mathbf{x}_{b,i}$, separated by the length of the fish ℓ_i . The source and the sink of each fish are defined to have the same volumetric flow rate, σ_i , to ensure the conservation of fluid by making the sinks consume exactly the same amount of fluid as the sources. For

simplicity, the source and sink (or “front” and “back”) of each fish are called its “features.”

Each fish is defined as an eight-dimensional vector consisting of four variables: $\vec{\mathbf{x}}_{c,i}$, the three-dimensional center-of-mass position vector, $\hat{\mathbf{n}}_i$, the three-dimensional orientation unit vector, ℓ_i , and σ_i . Together, the collection of each of these fish vectors at a given time forms the state vector $\vec{\mathbf{X}}$.

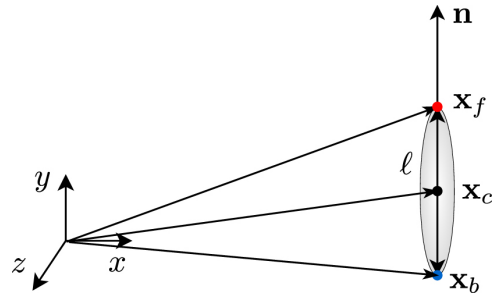


Figure 1: The model for a given swimmer visualized. Taken from Mabrouk and Floryan 2024.

In particular, $\vec{\mathbf{x}}_{c,i}$ and $\hat{\mathbf{n}}_i$ are dynamic and ℓ_i and σ_i are constant for each fish i . This formulism leaves the translational and rotational dynamics for each fish as the system of two coupled first-order differential equations concerning $\vec{\mathbf{x}}_{c,i}$ and $\hat{\mathbf{n}}_i$ respectively. With this, the derivatives of each are

*Corresponding author: dfloryan@uh.edu

defined as

$$\frac{d\vec{x}_{c,i}}{dt} = \frac{1}{2} \left(\frac{d\vec{x}_{f,i}}{dt} + \frac{d\vec{x}_{b,i}}{dt} \right) \quad (1)$$

$$\frac{d\hat{\mathbf{n}}_i}{dt} = \frac{\Delta\vec{v}_i + 2\lambda_i\hat{\mathbf{n}}_i}{\ell_i} \quad (2)$$

where

$$\lambda_i = \frac{\Delta\vec{v}_i \cdot \hat{\mathbf{n}}_i}{2} \quad (3)$$

is a Lagrange multiplier to ensure that ℓ_i is kept constant, and

$$\Delta\vec{v}_i = \frac{d\vec{x}_{f,i}}{dt} - \frac{d\vec{x}_{b,i}}{dt}. \quad (4)$$

With this, the translational and rotational dynamics for each fish are variably dependent on the translational derivatives of the source and sink for each fish, and this reduces the problem of modelling fish to an analogous problem of simple Laplacian particle dynamics based on treating the sources and sinks as individual particles with charge σ_i . Under this lens, the potential function resulting from a particle is the charge multiplied by Green's function for the three-dimensional Laplace equation,

$$\Phi_i(r) = \sigma_i G(r) = -\frac{\sigma_i}{4\pi r}. \quad (5)$$

However, as this model is designed for far-field interactions, it becomes unphysical for small distances, as infinitely small distances produce infinitely large potential. To solve this problem, Mabrouk and Floryan reformulated this potential with a regularization parameter ε defined as

$$\Phi_{i,\varepsilon}(r) = \Phi_i(r) \operatorname{erf}(r/\varepsilon). \quad (6)$$

From the potential functions, the velocity field resulting from any source or sink is defined as the gradient of the potential function, and for simplicity, the displacement-dependent vector $\vec{k}$ can be defined to represent the gradient of the Laplacian interaction kernel $1/r$,

$$\vec{k}(\vec{r}) = \nabla \left(-\frac{1}{r} \right) = \frac{\vec{r}}{r^3} \quad (7)$$

$$\vec{k}_\varepsilon(\vec{r}) = \nabla \left(-\frac{\operatorname{erf}(r/\varepsilon)}{r} \right) = \vec{k}(\vec{r}) \xi_\varepsilon(r) \quad (8)$$

where $\xi_\varepsilon(r)$ is the regularization coefficient

$$\xi_\varepsilon(r) = \operatorname{erf}(r/\varepsilon) - \frac{2r}{\varepsilon\sqrt{\pi}} \exp\left(-\frac{r^2}{\varepsilon^2}\right). \quad (9)$$

The velocity fields are therefore

$$\vec{u}_i(\vec{r}) = \nabla \Phi_i = \frac{\sigma_i}{4\pi} \vec{k}(\vec{r}) \quad (10)$$

$$\vec{u}_{i,\varepsilon}(\vec{r}) = \nabla \Phi_{i,\varepsilon} = \frac{\sigma_i}{4\pi} \vec{k}_\varepsilon(\vec{r}) \quad (11)$$

for the unregularized and regularized velocity fields respectively.

With the velocity fields well-defined, the velocity of a given source or sink α can be defined generally as

$$\frac{d\vec{x}_{\alpha,i}}{dt} = U_i \hat{\mathbf{n}}_i + \sum_{j \neq i}^N \frac{\sigma_j}{4\pi} \vec{K}_{\alpha,ij}. \quad (12)$$

where the first term is the *internal* velocity contribution and the second term is the *external* velocity contribution. In detail,

$$U_i = \frac{\sigma_i}{4\pi\ell_i^2} \quad (13)$$

is the self-propelled speed resulting from the interaction between any source-sink pair of a dipole, and the second term represents the sum of the velocity contribution to a source or sink from a dipole, where $\vec{K}_{\alpha,ij}$ represents the composite interaction vector between feature α of fish i with both the source f and the sink b of fish j ,

$$\vec{K}_{\alpha,ij} = \vec{k}(\vec{r}_{\alpha f}) - \vec{k}(\vec{r}_{\alpha b}). \quad (14)$$

where $\vec{r}_{\alpha\beta}$ is the displacement between feature α of fish i and feature β of fish j . In the case of regularization, the interaction function $\vec{k}$ is simply replaced with $\vec{k}_\varepsilon$.

TODO INCLUDE MISSING FIGURE SOURCE SINK INTERACTION

Derivative Computation

As stated previously, the modelling of fish is an N -body problem where the algorithmic complexity of computing a derivative is, through Brute-Force calculations, of order $\mathcal{O}(N^2)$. To reduce this computation time, two tree-based approximation algorithms are implemented: the Barnes-Hut algorithm and the Fast Multipole Method, of orders $\mathcal{O}(N \log N)$ and $\mathcal{O}(N)$ respectively.

Brute force computation and the Barnes-Hut approximation are implemented directly into the FINDS code-base, and as such, the user has the ability to switch between unregularized and regularized calculations with ease. However, the Fast-Multipole Method is computed externally using Green-gard et al.'s Fast Multipole Method library FMM3D, which will only compute the standard Laplacian gradient $\vec{u}_i$, so the Fast-Multipole Method cannot be used with regularization. Additionally, all three methods are CPU-parallelized using OpenMP.

The Barnes-Hut Approximation

As described by Barnes and Hut, the Barnes-Hut algorithm computes 3-dimensional pairwise interactions in $\mathcal{O}(N \log N)$ time by constructing a space-partitioning octree that clusters swimmers that are sufficiently far away from each test swimmer.

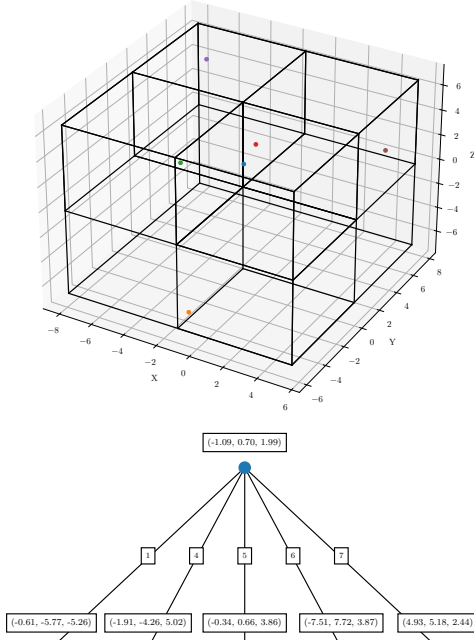


Figure 2: Top: An example octree constructed in three-dimensional space to partition five points, constructed with an arbitrary insertion order. Bottom: The same octree, represented two-dimensionally with the child octant indices.

Linear Octree Construction

FINDS implements a linear octree, or aligned arrays for each node parameter where each array index corresponds to a node in the tree. In particular, each node stores information on both the node itself and the data stored within this node. With regard to the node overall, it stores its center position, $\vec{x}_i$, the side length of the node, s_i , and indices to its child nodes. With regard to the data stored within this node, the node stores the number of swimmers that are being averaged out in this node n_i , the total volumetric flow rate,

$$\zeta_j = \sum_{j=0}^{n_i} \sigma_j \quad (15)$$

and weighted sums for the position, orientation, and length. Each sum is weighted by the volumetric flow rate (which can

be analogously thought of as the charge or mass in Coulombic or gravitational N -body dynamics), so that the average position, orientation, and length is produced by dividing by the total volumetric flow rate,

$$\overline{\vec{x}_{c,i}} = \zeta_j^{-1} \sum_{j=0}^{n_i} \vec{x}_{c,j} \sigma_j \quad (16)$$

$$\overline{\hat{\mathbf{n}}_i} = \zeta_j^{-1} \sum_{j=0}^{n_i} \hat{\mathbf{n}}_j \sigma_j \quad (17)$$

$$\overline{\ell_i} = \zeta_j^{-1} \sum_{j=0}^{n_i} \ell_j \sigma_j \quad (18)$$

where i in this context refers to the node index within the octree lists and j refers to the index of each swimmer stored within this node.

This architecture was chosen due to its traversal efficiency when sorted according to a space-filling curve, as this would keep physically close nodes close together in memory, which maximizes cache locality.

FIGURE MISSING ADD LINEAR OCTREE FIGURE

Construction of the Linear Octree begins by taking a system of swimmers $\vec{X}$ and computing vectors containing minimum and maximum values along each dimension for the system, or

$$\vec{x}_{min} = \langle \min \mathbf{x}, \min \mathbf{y}, \min \mathbf{z} \rangle \quad (19)$$

$$\vec{x}_{max} = \langle \max \mathbf{x}, \max \mathbf{y}, \max \mathbf{z} \rangle \quad (20)$$

where $\mathbf{x}$ contains all of the x -values for the position of every swimmer in the system, $\mathbf{y}$, the y -values, and $\mathbf{z}$, the z -values. Then, the center position of the root is defined as

$$\vec{x}_0 = \frac{\vec{x}_{min} + \vec{x}_{max}}{2} \quad (21)$$

and the side length for the root node is computed as the maximum component of their difference, or

$$s_0 = \max(\vec{x}_{max} - \vec{x}_{min}). \quad (22)$$

Next, each of the swimmers in the system need to be mapped to a space-filling curve to know what order to insert the nodes such that physically close swimmers are inserted in order. To accomplish this task, the 3-D position of each swimmer is mapped to a Morton curve localized to a relative origin point for the tree chosen as minimum position for the root node's bounding cube, or

$$\vec{x}_O = \vec{x}_0 - \frac{s_0}{2} \vec{1}_3. \quad (23)$$

Then, the position of each swimmer is truncated to an integer and a Morton number corresponding to this position is

produced by interleaving the binary coordinate values using bit-shifting operations. Note that this operation implicitly assumes that the three numbers going in are each a maximum of 21 bits such that the output Morton number is 64-bit, and this integer-sizing has significant consequences which will be discussed in the limitations section.

Then, a position-local array of swimmer indices is produced by first building an in-order list of indices $(0, 1, \dots, N)$ corresponding to the indices for each swimmer in $\vec{X}$, then arranging it according to the quick-sorted order of the corresponding Morton numbers for each swimmer position. Then, the linear octree is constructed recursively by defining each node region with the center position $\vec{x}_j$, side length s_j , tree depth d_j , and the corresponding index range of the swimmer array encompassing all of the swimmers that fall within this node. For example, for the root node, the center position is $\vec{x}_0$, the side length is s_0 , and the range is of enclosed swimmers is $[0, N]$. Then, for each swimmer contained within this range, the center-of-mass position $\vec{x}_{c,j}$, orientation $\hat{n}_j$, length ℓ_j , and volumetric flow rate σ_j are added accordingly to the corresponding weighted sums as stated previously, and then the weighted average source and sink positions are computed for this node based on the weighted average position, orientation, and length.

Next, the recursion is continued to child nodes, if any. If the range of swimmers that falls within this node is one or no swimmers, then the recursion ends here. If there are two or more swimmers within this octant, then this node cannot be a leaf, and this node is then subdivided into eight child octants. Each child octant is denoted by an index o from 0 to 7, and for each child octant, the range of node indices which should fall within the child octant is computed based on performing bit-shift operations on the associated Morton indices. This range is half-open, so if the start and end are equal, then no swimmers are located within the child node, and a new child node will only be added to the tree recursively if there are any swimmers located within the node. Otherwise, the child node is skipped.

In the case that a new child node is added, its side length $s_{j,o}$ is the side length of the current node divided by two,

$$s_{j,o} = s_j/2, \quad (24)$$

and its center position is computed based on its index, o , as

$$\vec{x}_{j,o} = \vec{x}_j + \frac{s_j}{4} \begin{pmatrix} f(o \& 1) \\ f(o \& 2) \\ f(o \& 4) \end{pmatrix} \quad (25)$$

where

$$f(x) = \begin{cases} 1 & \text{if } x = 1 \\ -1 & \text{if } x = 0 \end{cases}. \quad (26)$$

The binary representation of o can also be interpreted as a relative ordering on each axis between the center position of the parent node and the child octant. In order, the bits correspond with the z , y , and x -axes respectively, and 1 and 0 correspond with the center position of the child octant along a given dimension being greater than and less than or equal to that of the parent node for that dimension. Then, for each dimension, if the child position is less than or equal to the parent, then the child's center position is offset by $-s_j/4$. Otherwise, the child's center position is offset by $s_j/4$ along that axis. For example, the index $o = 5$ has binary representation 101, and this corresponds with $+s_j/4$ on the z -axis, $-s_j/4$ on the y -axis, and $+s_j/4$ on the x -axis.

Barnes-Hut Velocity Computation

Once constructed, the velocity contribution is computed in $\mathcal{O}(N \log N)$ time by performing a depth-first tree traversal on the linear octree. We can notate the test swimmer to compute the velocity for as a , the tree index for a given node as i , the weighted average (or composite) swimmer for index i as b_i , and the Barnes-Hut minimum approximation ratio supplied by the user as θ .

Beginning from the root node, if the current node is a leaf ($n_i = 1$) and the position for swimmer b_i is not approximately equal to the position of swimmer a (within a precision of 10^{-15} to account for floating-point errors) then the velocity contributions are computed immediately via equations 12 and 14, and the recursion is ended.

Otherwise, the size-to-distance ratio ϕ_i is computed for this node as

$$\phi_i = \frac{s_i}{d_{ab_i}} \quad (27)$$

where d is the distance from swimmer a to average swimmer b_i ,

$$d_{ab_i} = \left\| \vec{x}_{c,a} - \vec{x}_{c,b_i} \right\|. \quad (28)$$

Then, if $\phi_i < \theta$, then the composite swimmer b_i is said to be *close* to swimmer a , and the velocity contributions are computed immediately via equations 12 and 14 and the recursion ends. If $\phi_i \geq \theta$, then composite swimmer b_i is said to be *far* from swimmer a , and we recurse into each of the child nodes of node i .

The Fast Multipole Method (FMM)

As described by Greengard and Rokhlin, the fast multipole method similarly uses tree construction to compute the velocity field as the gradient of the potential field approximated via a multipole expansion of order $p(\epsilon)$ where ϵ (not to be confused with ε defined above) is the user-defined floating-point precision for the calculation. As stated previously, the implementation of the fast multipole method is not a part of

FINDS. Rather, the library FMM3D by Greengard et al. is used to compute the Laplacian gradient,

$$\vec{u}(\vec{x}) \approx \nabla \left(\sum_{j=1}^N \frac{\sigma_j}{4\pi \|\vec{x} - \vec{x}_j\|} \right) \quad (29)$$

for each source/sink using the function `lfmm3d_t_c_g`. In essence, this computation is based on treating the source and sink for each dipole as independent particles rather than members of a dipole, so to compute only the external interaction, each source/sink has to have its velocity corrected for double-counting the interaction with its pair. However, this is negligible compared to the overall calculation, as this is computed in $\mathcal{O}(1)$ time for each swimmer or $\mathcal{O}(N)$ time overall.

Numerical Integration

As stated above, equations 1 and 2 form a system of coupled first-order ordinary differential equations, and as such, these can be integrated using conventional first-order integration schemes like Runge-Kutta methods. As such, FINDS implements a variety of variable-order adaptive and non-adaptive methods of various order (as seen in table 1).

Method	Order	Adaptive?
Forward Euler	1	No
RK2	2	No
RK4	4	No
RK5	5	No
RK6	6	No
Bogacki-Shampine RK3(2)	3	Yes, 2nd Order
Dormand-Prince RK5(4)	5	Yes, 4th Order

Table 1: List of the numerical integrators implemented in FINDS. If it is adaptive, the error computation order is also included.

Given a user-defined evaluation time step Δt and an end time T , the goal of the integration module is to return the associated system states at the times $\{0, \Delta t, 2\Delta t, \dots, T_f\}$ where $T_f = T$ for adaptive integrators and $T_f = \text{floor}(T/\Delta t)$ for non-adaptive integrators.

To do so, the simulation integrates over the time domain of $t \in [0, T_f]$. Non-adaptive integrators will hold the computation time-step δt_i (where, in this context, i is the time-step index) constant as the evaluation time-step, $\delta t_i \equiv \Delta t$, and adaptive integrators will compute, for every time-state $\vec{X}_i$ and δt_i , the advanced state $\vec{X}_{i+1}$ and a new computation time-step δt_{i+1} , computed as

$$\delta t_{i+1} = \delta t_i \left(\frac{\xi_i}{E_i} \right)^{\frac{1}{p}} \quad (30)$$

where ξ_i is the tolerance, computed as

$$\xi_i = \gamma + \mu \max(\|\vec{X}_{i+1}\|, \|\vec{X}_i\|) \quad (31)$$

where γ and μ are the user-specified absolute and relative tolerance respectively, and E_i is the error of time-step i , computed as

$$E_i = \|\vec{X}_{p,i+1} - \vec{X}_{q,i+1}\| \quad (32)$$

where p is the computation order and q is the error order (for example, for RK5(4), $p = 5$ and $q = 4$) and $\vec{X}_{p,i}$ refers to the p -th order solution for step i .

Database Management

The evaluation times $\{0, \Delta t, 2\Delta t, \dots, T_f\}$ and their corresponding system states $\{\vec{X}_1, \vec{X}_2, \dots, \vec{X}_{T_f}\}$ are written out to an HDF5 file in the following format:

Initial System Creation

Running Simulations from Configuration Files

Validation

FINDS was validated

Performance Evaluation

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Examples

Limitations

Barnes-Hut is using a monopole representation for dipole interactions, and this is slightly incorrect (but mostly fine)

Morton numbers are only 64-bit, so position dimension values can only be 21 bit, and think what the max double that can be represented in 21 bit is

FMM has incredibly high overhead

Future Work

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Acknowledgements

In the development of FINDS, Alvin Ng's `grav_sim` was used extensively as a reference for implementing a fast N -body simulator in C, particularly in implementing of the Barnes-Hut algorithm. In addition, the direct advice of Alvin Ng was instrumental to the implementation of linear octrees and parallelization.

Furthermore, generative artificial intelligence (Anthropic's Sonnet 5 Claude, OpenAI's ChatGPT-5.5 and Google's Gemini 3.3) were used heavily for initial code generation before being thoroughly tested and validated to ensure correctness. Artificial intelligence was not used in developing the content of this report whatsoever, including the abstract, section text, figures, and captions.

References

- [1] Mohamed Niged Mabrouk and Daniel Floryan. "Group cohesion and passive dynamics of a pair of inertial

swimmers with three-dimensional hydrodynamic interactions". In: *Bioinspiration and Biomimetics* 20.1 (Nov. 2024), p. 016014. DOI: [10.1088/1748-3190/ad936d](https://doi.org/10.1088/1748-3190/ad936d). URL: <https://doi.org/10.1088/1748-3190/ad936d>.

- [2] Mohamed Niged Mabrouk and Daniel Floryan. "Effects of symmetry and hydrodynamics on the cohesion of groups of swimmers". In: *Bioinspiration and Biomimetics* 20.6 (Oct. 2025), p. 066005. DOI: [10.1088/1748-3190/ae0bd9](https://doi.org/10.1088/1748-3190/ae0bd9). URL: <https://doi.org/10.1088/1748-3190/ae0bd9>.

Appendix